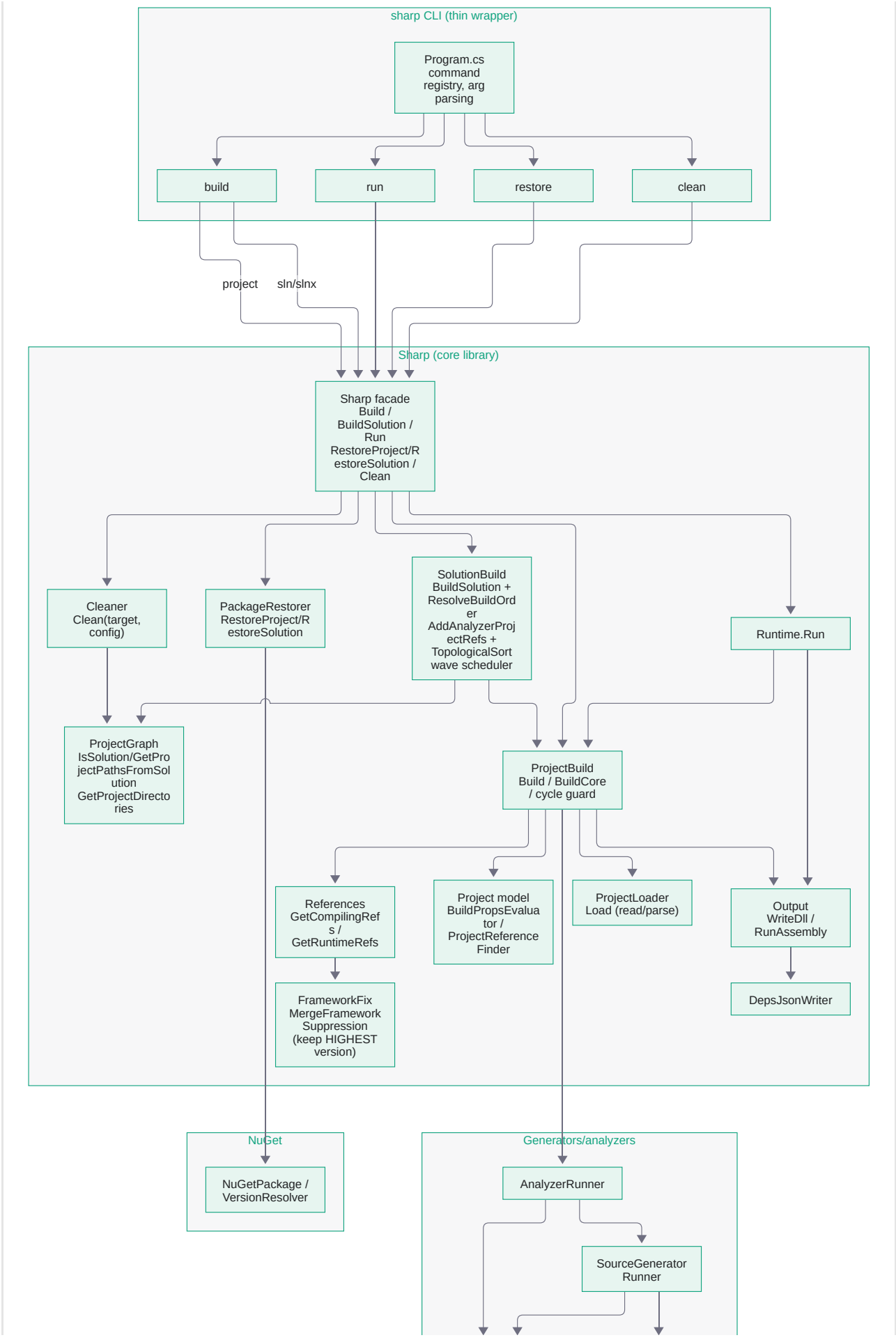
ARCHITECTURE NOTE

Sharp Compiler Architecture — v1.1 (unified)

One diagram showing the hierarchy: CLI is a thin wrapper over the `Sharp` core library. The core library exposes the same command surface (`Build` , `BuildSolution` , `Run` , `RestoreProject` / `RestoreSolution` , `Clean`) so it can be consumed as a library without depending on the CLI. Implementation lives in intent-based vertical slices.


sharp CLI (thin wrapper)
Program.cs command registry, arg parsing
build
run
restore
clean
project
sln/slnx
Sharp (core library)
Sharp facade Build / BuildSolution / Run RestoreProject/RestoreSolution / Clean
Cleaner Clean(target, config)
PackageRestorer RestoreProject/RestoreSolution
SolutionBuild BuildSolution + ResolveBuildOrder AddAnalyzerProjectRefs + TopologicalSort wave scheduler
Runtime.Run
ProjectGraph IsSolution/GetProjectPathsFromSolution GetProjectDirectories
ProjectBuild Build / BuildCore / cycle guard
References GetCompilingRefs / GetRuntimeRefs
Project model BuildPropsEvaluator / ProjectReference Finder
ProjectLoader Load (read/parse)
Output WriteDll / RunAssembly
FrameworkFix MergeFramework Suppression (keep HIGHEST version)
DepsJsonWriter
NuGet
NuGetPackage / VersionResolver
Generators/analyzers
AnalyzerRunner
SourceGenerator Runner



Notes:

- `Clean` moved into the core (`Cleaner.Clean`) so both CLI and library consumers can clean project outputs. CLI only formats/parses; the logic lives under `Sharp.Cleaner` .
- Restore is part of the core (`PackageRestorer`) and exposed via the `Sharp` facade; CLI delegates. Diagram shows restore under the core, not a separate detached box.
- Solution builds use wave scheduling (Kahn) with analyzer project references (self-edges skipped); single-project builds have a cycle guard and return a failed `BuildResult` on resolution errors.
- Reference dedup keeps the **highest** assembly version on simple-name clashes (`FrameworkFix`).